

题目	学号	姓名	班级	日期
综合应用设计	2021302565 2021302598	李旭辉 李欣鸿	10012102 10042101	12.18

- 需求分析
 - 需求分析
 - 数据字典
 - 数据流图
- 概念设计
- 逻辑设计
 - 关系模式
 - 表设计
- 功能设计
 - 用户信息管理
 - 商品管理
 - 财务管理
 - 进货管理
 - 库存管理
- 使用说明
 - 环境配置
 - 文件结构
 - 操作步骤
- 源代码
- 问题与解决方案
- 总结与反思

需求分析

需求分析

本应用主要面向医药产品的零售，批发等相关的企业以及买家，分析可得需求有：

针对用户：

- 购物管理：能够进行药品的选购及退货。

针对企业：

- 基础信息管理：能够统计药品信息，员工信息，客户信息，供应商信息等。
- 进货管理：能够进行药品入库登记，入库查询与修改等操作。
- 库房管理：能够查询、盘点库存，处理被退货的商品等。
- 销售管理：能够进行销售登记，销售退货，销售查询等。
- 财务统计：能够统计当日、当月统计等的财务信息。

数据字典

1. 药品信息

数据项名	id
说明	BIGHT
类型	药品编号

数据项名	drug_name
说明	药品名
类型	varchar()

数据项名	product_id
说明	生产批号
类型	VARCHAR

数据项名	product_place
说明	产地
类型	VARCHAR

数据项名	type
说明	药品类别
类型	VARCHAR

数据项名	in_price
说明	进价
类型	DOUBLE

数据项名	single_price
说明	单价
类型	DOUBLE

数据项名	discount
说明	会员折扣
类型	DOUBLE

数据项名	package
说明	库存量
类型	INT

数据项名	size
说明	规格
类型	VARCHAR

数据项名	prodate_date
说明	生产日期
类型	DATE

数据项名	valid_date
说明	有效期
类型	DATE

具体如下：

Field Types									
#	Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale	
1	id	inventory_management...	demo_drugs	BIGINT	binary	20	2	0	
2	drug_name	inventory_management...	demo_drugs	VARCHAR	utf8mb4	50	12	0	
3	product_id	inventory_management...	demo_drugs	VARCHAR	utf8mb4	100	7	0	
4	product_place	inventory_management...	demo_drugs	VARCHAR	utf8mb4	50	6	0	
5	type	inventory_management...	demo_drugs	VARCHAR	utf8mb4	50	1	0	
6	in_price	inventory_management...	demo_drugs	DOUBLE	binary	22	-26	31	
7	single_price	inventory_management...	demo_drugs	DOUBLE	binary	22	-26	31	
8	discount	inventory_management...	demo_drugs	DOUBLE	binary	22	-28	31	
9	package	inventory_management...	demo_drugs	INT	binary	11	3	0	
10	size	inventory_management...	demo_drugs	VARCHAR	utf8mb4	50	1	0	
11	pro_date	inventory_management...	demo_drugs	DATE	binary	10	10	0	
12	valid_date	inventory_management...	demo_drugs	DATE	binary	10	10	0	

2. 客户信息

数据项名	id
说明	客户编号
类型	BIGINT

数据项名	name
说明	客户姓名
类型	VARCHAR

数据项名	emali
说明	邮箱地址
类型	VARCHAR

具体如下：

Field Types								
#	Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale
1	id	inventory_management...	demo_member	BIGINT	binary	20	2	0
2	id_client	inventory_management...	demo_member	VARCHAR	utf8mb4	16	3	0
3	name	inventory_management...	demo_member	VARCHAR	utf8mb4	50	12	0
4	email	inventory_management...	demo_member	VARCHAR	utf8mb4	32	27	0

3. 供应商信息

数据项	id
说明	供应商id
类型	BIGINT

数据项名	供应商名称
说明	邮箱地址
类型	VARCHAR

数据项名	联系人
说明	邮箱地址
类型	VARCHAR

数据项名	联系方式
说明	邮箱地址
类型	VARCHAR

数据项名	城市
说明	邮箱地址
类型	VARCHAR

Field Types								
#	Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale
1	id	inventory_management...	demo_provider	BIGINT	binary	20		0
2	pro_name	inventory_management...	demo_provider	VARCHAR	utf8mb4	50		0
3	linkman	inventory_management...	demo_provider	VARCHAR	utf8mb4	50		0
4	linkWay	inventory_management...	demo_provider	VARCHAR	utf8mb4	50		0
5	city	inventory_management...	demo_provider	VARCHAR	utf8mb4	50		0
6	provider_id	inventory_management...	demo_provider	INT	binary	11		0

4. 员工信息

数据项名	worker_id
说明	员工编号
类型	BIGINT

数据项名	worker_name
说明	员工姓名
类型	VARCHAR

数据项名	telephone
说明	电话
类型	VARCHAR

数据项名	increment_id
说明	关联用户
类型	VARCHAR

Field Types								
#	Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale
1	id	inventory_management...	demo_provider	BIGINT	binary	20	0	0
2	pro_name	inventory_management...	demo_provider	VARCHAR	utf8mb4	50	0	0
3	linkman	inventory_management...	demo_provider	VARCHAR	utf8mb4	50	0	0
4	linkWay	inventory_management...	demo_provider	VARCHAR	utf8mb4	50	0	0
5	city	inventory_management...	demo_provider	VARCHAR	utf8mb4	50	0	0
6	provider_id	inventory_management...	demo_provider	INT	binary	11	0	0

5. 销售信息

数据项名	io_number
说明	订单编号
类型	INT

数据项名	drug_id
说明	药品编号
类型	INT

数据项名	number
说明	出售数量
类型	INT

数据项名	date
说明	日期
类型	DATE

数据项名	money
说明	销售额
类型	DOUBLE

数据项名	user_increment
说明	客户编号
类型	INT

数据项名	type
说明	药品类型
类型	VARCHAR

Field Types								
#	Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale
1	id	inventory_management...	demo_in_out	BIGINT	binary	20	5	0
2	io_number	inventory_management...	demo_in_out	INT	binary	11	4	0
3	drug_id	inventory_management...	demo_in_out	INT	binary	11	7	0
4	number	inventory_management...	demo_in_out	INT	binary	11	1	0
5	date	inventory_management...	demo_in_out	DATETIME	binary	26	20	6
6	money	inventory_management...	demo_in_out	DOUBLE	binary	22	-26	31
7	type	inventory_management...	demo_in_out	VARCHAR	utf8mb4	50	1	0
8	user_increment	inventory_management...	demo_in_out	INT	binary	11	1	0

6. 入库信息

数据项名	in_id
说明	入库编号
类型	VARCHAR

数据项名	io_number
说明	流水单号
类型	VARCHAR

数据项名	provider_id
说明	供应商编号
类型	INT

数据项名	number
说明	单品数量
类型	INT

数据项名	name
说明	药品类型
类型	VARCHAR

Field Types									
#	Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale	
1	id	invetory_management...	demo_in_document	BIGINT	binary	20		0	0
2	in_id	invetory_management...	demo_in_document	VARCHAR	utf8mb4	50		0	0
3	io_number	invetory_management...	demo_in_document	INT	binary	11		0	0
4	provider_id	invetory_management...	demo_in_document	INT	binary	11		0	0
5	number	invetory_management...	demo_in_document	INT	binary	11		0	0
6	name	invetory_management...	demo_in_document	VARCHAR	utf8mb4	50		0	0

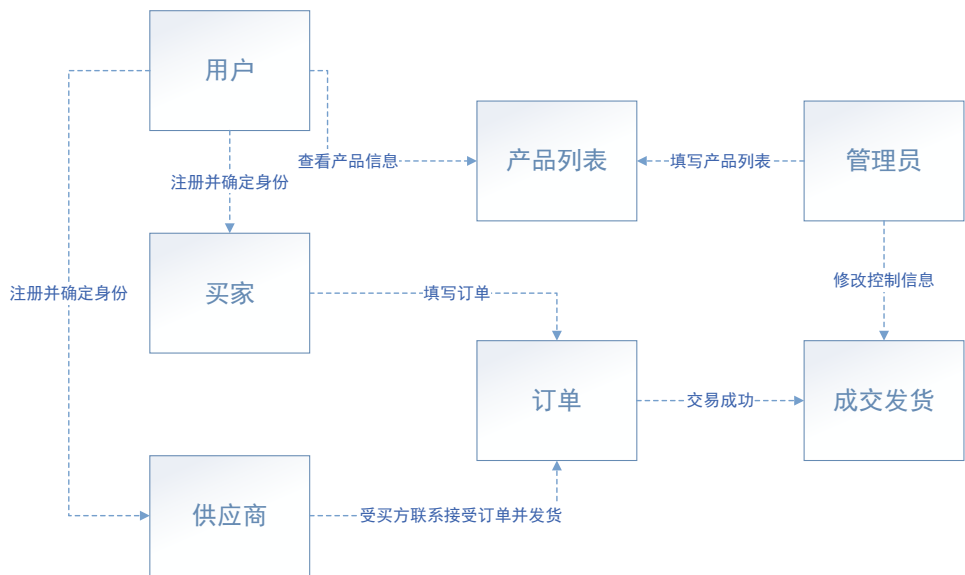
7. 库存信息

数据项名	drug_id
说明	药品编号
类型	BIGINT

数据项名	drug_name
说明	药品名称
类型	VARCHAR

数据项名	package
说明	药品库存
类型	VARCHAR

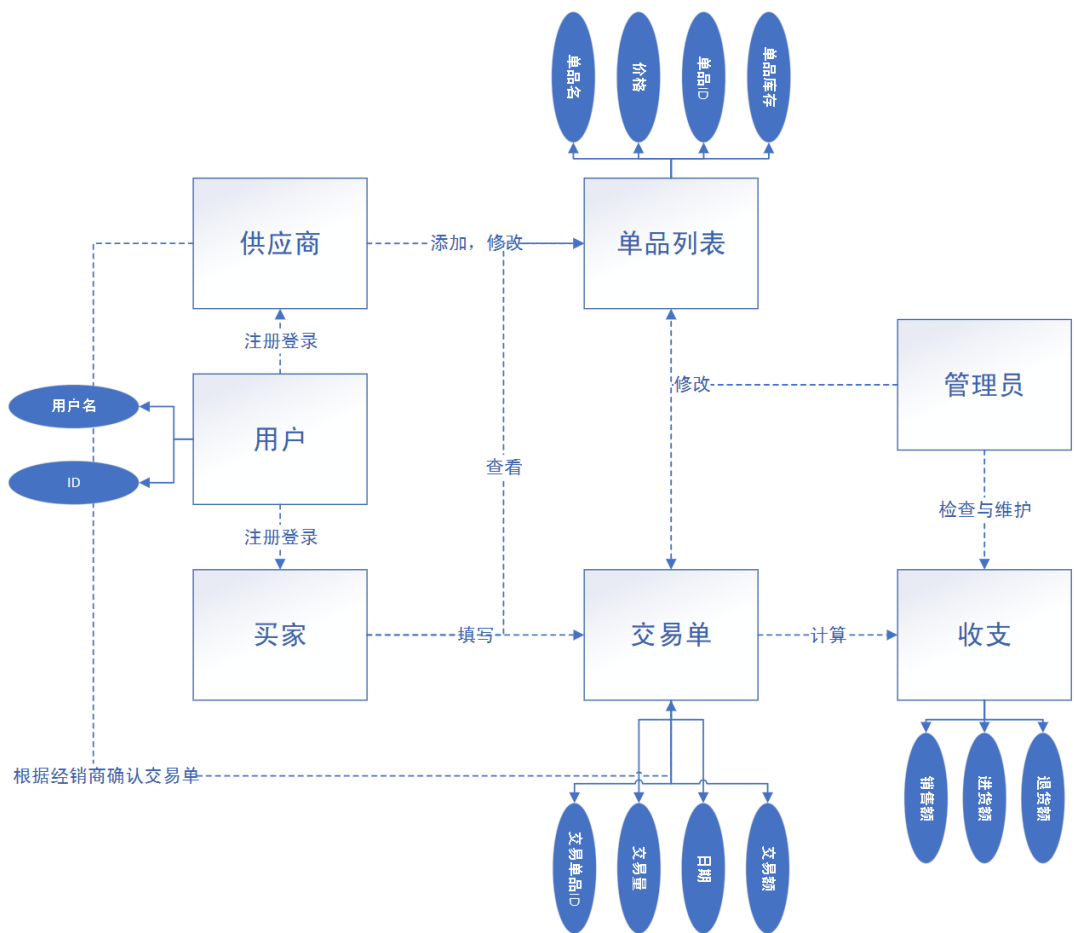
数据流图



概念设计

E-R图设计

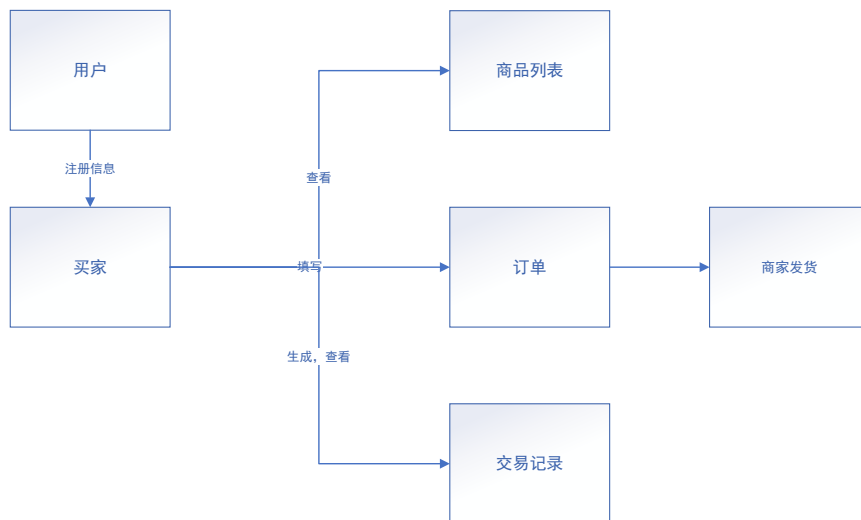
涉及实体主要包括：买家、供应商、单品列表、交易单、管理员和收支。



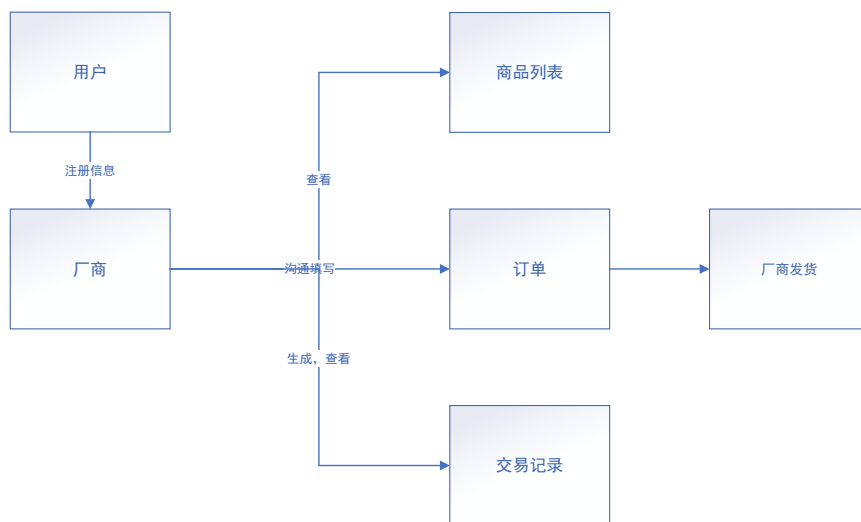
逻辑设计

关系模式

1. 买家即用户端:



2. 商家即服务端



表设计

除开django框架的相关权限和日志表格外，还涉及多张表：

▶	auth_user_user_permissions
▶	demo_drugs
▶	demo_in_document
▶	demo_in_out
▶	demo_manager
▶	demo_member
▶	demo_provider
▶	demo_sales_returning
▶	demo_sell_manage
▶	demo_user
▶	demo_worker
▶	django_admin_log

将数据库在本地导入可以看到：

1. 账号表：

能够登陆此系统的账号

id	user_increment	user_name	pwd	type	balance
1	1	lixuhui	qwe20031221	admin	5147
2	2	root	root666	admin	5000
NULL	NULL	NULL	NULL	NULL	NULL

2. 药品表：

提供药品的各种信息

id	drug_name	product_id	product_place	type	in_price	single_price	discount	package	size	pro_date	valid_date
12	毒药123	1230032	缅甸	1	25777	123.5	1.2	98	0	2023-12-01	2044-12-01
13	阿莫西林	1230032	西安	1	25777	25.5	1.2	102	2	2021-12-01	2044-12-01
15	阿莫西林	1230032	西安	1	25777	25.5	1.2	102	2	2021-12-01	2044-12-01
16	莲花清瘟	12302	北京	1	25777	15.8	1.2	102	2	2019-12-01	2044-12-01
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

3. 订单表

id	io_number	drug_id	number	date	money	type	user_increment
10002	1807	1230032	1	2023-11-28 23:58:19.611270	123.5	1	5
10003	1373	1230032	1	2023-11-28 23:59:05.475972	123.5	1	5
10004	1218	1230032	1	2023-11-29 00:00:38.382055	123.5	1	5
10005	1378	1230032	1	2023-11-29 00:01:06.102648	123.5	1	5
10007	1897	1230032	1	2023-11-29 00:15:43.247091	25.5	1	5
10008	1763	1230032	1	2023-12-06 23:02:09.661026	5123.5	-1	5
10009	1446	1230032	2	2023-12-06 23:02:44.425487	4876	1	5
10010	1174	1230032	1	2023-12-06 23:04:13.781508	4999.5	-1	5
10011	1788	1230032	1	2023-12-06 23:04:51.150340	5024.5	-1	5
10012	1137	1230032	1	2023-12-06 23:06:27.025640	5147.5	-1	5
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

4. 进货表

Result Grid						
Filter Rows:						
	id	in_id	io_number	provider_id	number	name
*	NULL	NULL	NULL	NULL	NULL	NULL

5. 用户表

在该系统上注册，能够购买商品的用戶

Result Grid				
Filter Rows:				
	id	id_client	name	email
▶	47	115	lixuhui	lixuhui123@mail.nwpu.edu.cn
	48	116	lixuhui	lixuhui123@mail.nwpu.edu.cn
	49	225	万海东 123	wanhd@nwpu.edu.cn
*	NULL	NULL	NULL	NULL

6. 供应商表

Result Grid						
Filter Rows:						
	id	pro_name	linkman	linkWay	city	provider_id
*	NULL	NULL	NULL	NULL	NULL	NULL

7. 退货表

Result Grid

Filter Rows:

Edit:

	id	id_returning	id_sell_id	io_number_id	number
*	NULL	NULL	NULL	NULL	NULL

8. 员工表

<

Result Grid





Filter Rows:

Edit: 



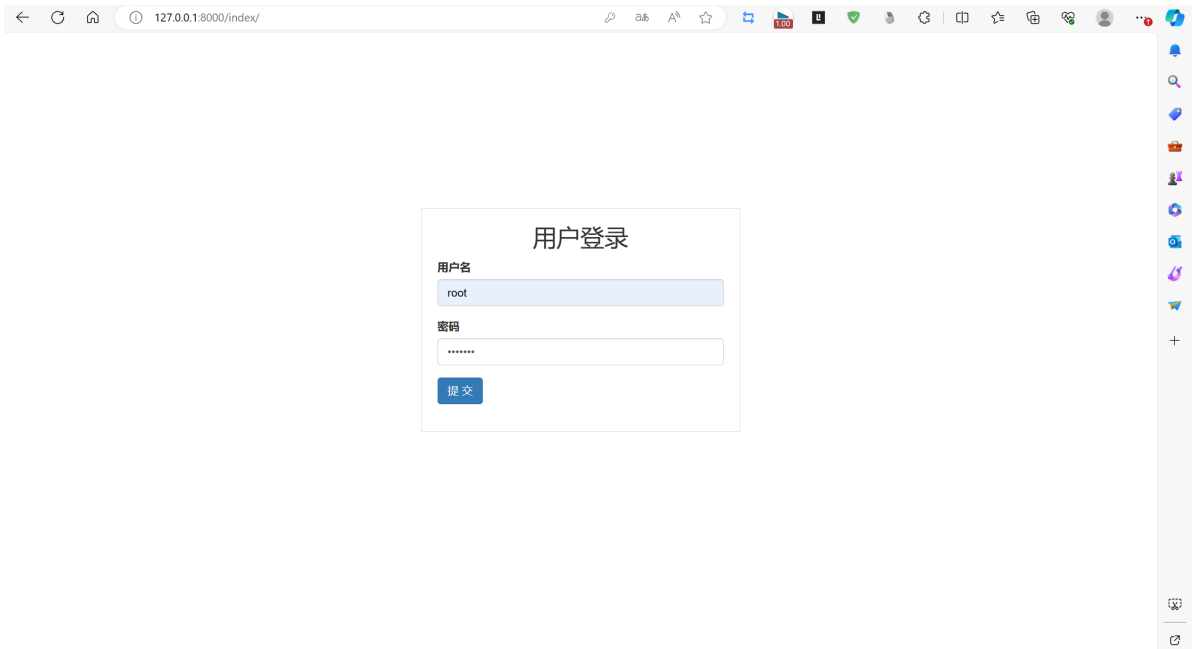


Export/Imp

	id	worker_id	worker_name	telephone	user_increment_id
*	NULL	NULL	NULL	NULL	NULL

功能设计

本项目可使用不同的账号登录：



登录进入系统后可以看到主界面：



用户信息管理

在主页点击用户信息管理可以看到当前的供应商信息，用户信息和员工信息：

小辉科技Link信息管理(供应商-客户)商品管理财务管理选项SearchSubmit来一个打赏吧亲登录状态

添加供应商信息

ID	供应商名称	联系人	联系方式	城市	供应商ID	操作
----	-------	-----	------	----	-------	----

添加用户

ID	姓名	联系方式	操作
43	万海东	wanhd@nwpu.edu.cn	删除编辑
47	lixuhui	lixuhui123@mail.nwpu.edu.cn	删除编辑
48	lixuhui	lixuhui123@mail.nwpu.edu.cn	删除编辑
49	万海东123	wanhd@nwpu.edu.cn	删除编辑

添加公司员工信息

ID	员工号	员工姓名	电话	关联用户编号	操作
----	-----	------	----	--------	----

可以点击对应的按钮对表项进行添加修改或删除：

添加供应商

ID

供应商名称

联系人

联系方式

城市

保存

修改部门

ID

姓名

邮箱

保存

商品管理

在主页点击商品管理可以进入商品界面，界面中有三种药品的大图展示，用户和公司余额以及当前所有商品的信息：



阿莫西林

价格

购买 退货 进货



肾宝片

价格

购买 退货 进货



莲花清瘟

价格

购买 退货 进货

新建单品

用户的余额是：
公司的总余额是：

药品名	生产批号	产地	药品类别	进价	单价	会员折扣	库存量	规格	生产日期	有效期	选项
毒药123	1230032	缅甸	1	25777.0	123.5	1.2	100	0	Dec. 1, 2023	Dec. 1, 2044	删除 编辑 下单 退货
阿莫西林	1230032	西安	1	25777.0	25.5	1.2	102	2	Dec. 1, 2021	Dec. 1, 2044	删除 编辑 下单 退货
阿莫西林	1230032	西安	1	25777.0	25.5	1.2	102	2	Dec. 1, 2021	Dec. 1, 2044	删除 编辑 下单 退货
莲花清瘟	12302	北京	1	25777.0	15.8	1.2	102	2	Dec. 1, 2019	Dec. 1, 2044	删除 编辑 下单 退货

点击新建单品可以向表中添加新的商品信息：

添加商品

药品名

商品号

产地

药品类别

感冒药

进价

单价

会员折扣

库存量

规格

生产日期

有效期

保存

点击修改可以修改原表中的信息并保存：

修改药品信息

毒药123

1230032

缅甸

1

感冒药

25777.0

123.5

1.2

100

0

Dec. 1, 2023

Dec. 1, 2044

保存

点击下单可以生成订单，请用户确认是否购买：

仔细查看你的订单信息

药品名	生产批号	产地	药品类别	进价	单价	会员折扣	库存量	规格	生产日期	有效期	选项
毒药123	1230032	缅甸	1	25777.0	123.5	1.2	100	0	Dec. 1, 2023	Dec. 1, 2044	<button>删除</button> <button>编辑</button>

亲准备下单咯

收支编号	药品	关联用户编号	数量	日期	付款后总额	类别	操作
1446	1230032	5	2	Dec. 6, 2023, 11:02 p.m.	4876.0	1	<button>取消订单</button> <button>修改数量</button>

确认下单

点击退货可以检测已有订单并确认是否退货：

你舍不得我了吗

药品名	生产批号	产地	药品类别	进价	单价	会员折扣	库存量	规格	生产日期	有效期	选项
毒药123	1230032	缅甸	1	25777.0	123.5	1.2	98	0	Dec. 1, 2023	Dec. 1, 2044	<button>删除</button> <button>编辑</button>

球球你不要退货

收支编号	药品	关联用户编号	数量	日期	总额	类别	操作
1174	1230032	5	1	Dec. 6, 2023, 11:04 p.m.	4999.5	-1	<button>取消退货</button> <button>修改数量</button>

确认退货

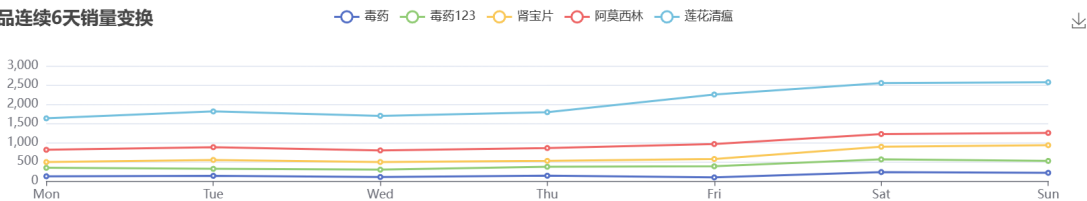
财务管理

在主页点击财务管理进入财务页面，可以看到各药品近六天的销量变化折线图，员工业绩考核饼状图以及药品销量饼状图：

当前您的余额是: 5000 ¥

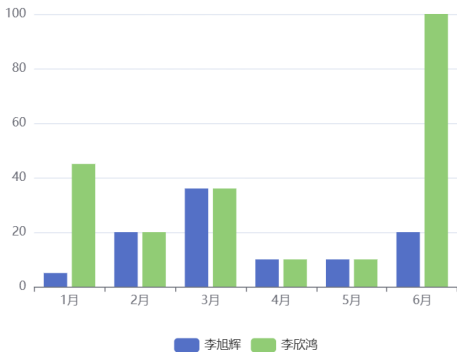
折线图

药品连续6天销量变换



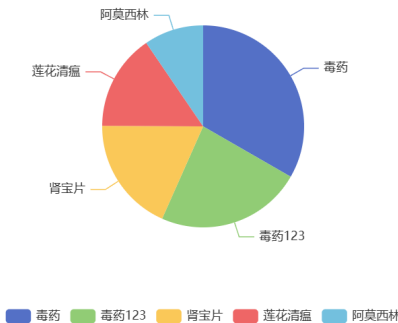
柱状图

公司上年度业绩考察表



饼状图

药品收入来源分布



该界面中还有近一个月中所有销售订单的表格以及进过货的药品信息。点击删除和编辑同样可以对表项进行操作：

销售单号

收支编号	药品	关联用户编号	数量	日期	公司全部财产	类别	
1807	1230032	5	1	Nov. 28, 2023, 11:58 p.m.	123.5	1	删除 编辑
1373	1230032	5	1	Nov. 28, 2023, 11:59 p.m.	123.5	1	删除 编辑
1218	1230032	5	1	Nov. 29, 2023, midnight	123.5	1	删除 编辑
1378	1230032	5	1	Nov. 29, 2023, 12:01 a.m.	123.5	1	删除 编辑
1897	1230032	5	1	Nov. 29, 2023, 12:15 a.m.	25.5	1	删除 编辑

入库单号

入库ID	供应商编号	单品数量	流水单号	类别
------	-------	------	------	----

进货管理

在主界面点击进货管理可以进入进货界面，可以看到所有药品的信息：

创建进货单品

药品名	生产批号	产地	药品类别	进价	单价	会员折扣	库存量	规格	生产日期	有效期	选项
毒药123	1230032	缅甸	1	25777.0	123.5	1.2	100	0	Dec. 1, 2023	Dec. 1, 2044	删除 编辑 进货 退货(退给经销商)
阿莫西林	1230032	西安	1	25777.0	25.5	1.2	102	2	Dec. 1, 2021	Dec. 1, 2044	删除 编辑 进货 退货(退给经销商)
阿莫西林	1230032	西安	1	25777.0	25.5	1.2	102	2	Dec. 1, 2021	Dec. 1, 2044	删除 编辑 进货 退货(退给经销商)
莲花清瘟	12302	北京	1	25777.0	15.8	1.2	102	2	Dec. 1, 2019	Dec. 1, 2044	删除 编辑 进货 退货(退给经销商)

点击创建进货单品还可以添加全新种类的药品：

添加商品

药品名

进价

生产日期

商品号

单价

有效期

产地

会员折扣

保存

药品类别

库存量

感冒药

规格

库存管理

在主界面中点击选项-库存管理可以查看现有库存：

库存盘点

单品号	商品名称	库存量	操作
1230032	毒药123	100	进货 编辑-修改
1230032	阿莫西林	102	进货 编辑-修改
1230032	阿莫西林	102	进货 编辑-修改
12302	莲花清瘟	102	进货 编辑-修改

使用说明

环境配置

python 3.6

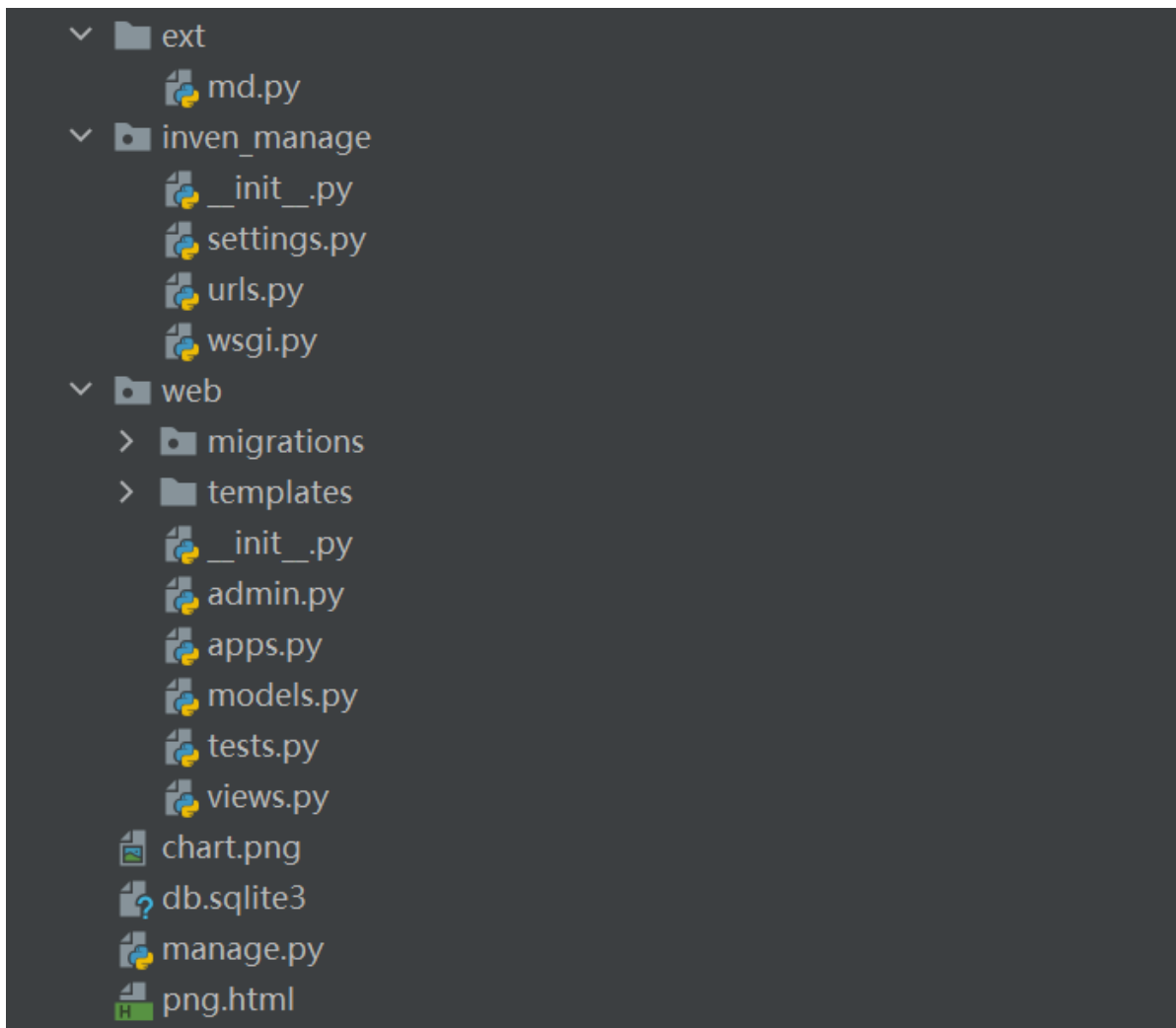
MySql 8.0

matplotlib

文件结构

```

▼ inven_manage
  ▼ demo
    ▼ migrations
      0001_initial.py
      0002_member_delete_memeber.py
      0003_alter_drugs_package_alter_drugs_sup_id_and_more.py
      0004_rename_productor_id_drugs_product_id_and_more.py
      0005_remove_drugs_sup_id_alter_in_out_date_and_more.py
      0006_alter_in_out_user_increment.py
      0007_alter_in_out_user_increment.py
      0008_alter_drugs_pro_date_alter_drugs_valid_date_and_more.py
      0009_alter_in_document_io_number_and_more.py
      0010_in_document_number.py
      0011_sales_returning_number.py
      0012_user_balance.py
      0013_in_document_name.py
      __init__.py
    ▼ sql
      costum.sql
  > static
  > templates
  __init__.py
  admin.py
  apps.py
  models.py
  tests.py
  views.py
```



操作步骤

首次运行本系统需要按如下步骤操作：

1. 导入附带的数据库 `Dump20231129.sql`
2. 在 `inven_manage/setting.py` 中将数据库账号和密码修改为本地账号密码。



3. 打开终端，运行 `python manage.py migrate` 命令。
4. 运行 `python manage.py runserver` 命令获得本地系统链接 `http://127.0.0.1:8000/`

```
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
December 17, 2023 - 22:31:58
Django version 3.2.23, using settings 'inven_manage.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

5. 在浏览器中打开 `http://127.0.0.1:8000/index` 进入系统。

源代码

部分关键代码如下：

viewe.py

```
from django.shortcuts import render, redirect, HttpResponseRedirect
from django.http import JsonResponse
from demo import models
from django import forms
import random
import datetime
import matplotlib.pyplot as plt

# Create your views here.
class RoleForm(forms.Form):
    user = forms.CharField(
        label="用户名",
        # widget=forms.Textarea(attrs={"class": "form-control"})
    )
    password = forms.CharField(
        label="密码",
        widget=forms.PasswordInput(attrs={"class": "form-control"}, render_value=True)
    )
    type = forms.ChoiceField(
        choices=[(1, "供应商"), (2, "客户"), (3, "员工"), (4, "经理"), (5, "管理员"), ],
        widget=forms.Select(attrs={"class": "form-control"})
    )

def add_user(request):
    if request.method == "GET":
        form = RoleForm(initial={'user': "your name", 'password': "123123"})
        return render(request, 'add_user.html', {"form": form})

    from1 = RoleForm(data=request.POST)
    if from1.is_valid():
        print("成功", from1.cleaned_data)
        return HttpResponseRedirect("成功")
    else:
```

```

        print("失败", from1.errors)
        return render(request, 'add_user.html')

def index(request):
    if request.method == "GET":
        return render(request, "index.html")
    else:
        # 请求体中获取数据, 校验{user ... pwd}
        username = request.POST.get('user_name')
        password = request.POST.get('pwd')
        print(username)
        print(password)
        # 去数据库校验用户名和密码的合法性
        # 成功--跳转后台管理界面
        user_object = models.user.objects.filter(user_name=username,
pwd=password).first()
        # print("用户名:", user_object.user_name)
        # print("密码:", user_object.pwd)
        if user_object:
            request.session["info"] = user_object.user_name
            return redirect('/real/')
        else:
            return render(request, "index.html", {"error": "用户名或密码错误"})

def real(request):
    # 判断是否登录, 否则跳转到登录页面
    print("hello world")
    info_dict = request.session.get("info")
    print("info_dict:", info_dict)

    queryset = [
        {"id": 1, "phone": 1888823888, "city": "北京"},
        {"id": 2, "phone": 1358888888, "city": "北京"},
        {"id": 3, "phone": 1888882448, "city": "上海"},
        {"id": 4, "phone": 1882488888, "city": "北京"},
    ]

    return render(request, "index_real.html", {"data": queryset, "info_dict":
info_dict})

def depart(request):
    # ORM 操作 增加
    # add
    # models.member.objects.create(id_client="123", name="李旭辉",
email="123@live.com")
    queryset = models.member.objects.all().order_by("id")
    queryset1 = models.provider.objects.all().order_by("id")
    queryset2 = models.worker.objects.all().order_by("id")
    # 这里对员工的删改纯属冗余, 不必考虑
    return render(request, 'depart.html', {'queryset': queryset, 'queryset1':
queryset1, 'queryset2': queryset2})

```

```
def add_member(request):
    if (request.method == "GET"):
        return render(request, 'add_member.html')
    id_client = request.POST.get("id_client")
    name = request.POST.get("name")
    email = request.POST.get("email")

    models.member.objects.create(id_client=id_client, name=name, email=email)

    # 跳转展示
    return redirect("/depart/")


def delete_member(request):
    id_dle = request.GET.get('id')
    print(id_dle)
    models.member.objects.filter(id=id_dle).delete()

    return redirect("/depart/")


def edit_member(request):
    # 列表页面跳转时的
    if request.method == "GET":
        id_client = request.GET.get("id")
        member_obj = models.member.objects.filter(id_client=id_client).first()
        return render(request, 'edit_member.html', {'member_obj': member_obj})
    else:
        member_id = request.GET.get("id")
        name = request.POST.get("name")
        email = request.POST.get("email")
        models.member.objects.filter(id=member_id).update(name=name, email=email)
        return redirect("/depart/")


def goods(request):
    queryset = models.drugs.objects.all()
    return render(request, 'goods.html', {'queryset': queryset})


def add_goods(request):
    if (request.method == "GET"):
        return render(request, 'add_goods.html')
    info_dict = request.session.get("info")
    drug_name = request.POST.get("drug_name")
    product_id = request.POST.get("product_id")
    product_place = request.POST.get("product_place")
    type = request.POST.get("type")
    in_price = request.POST.get("in_price")
    single_price = request.POST.get("single_price")
    discount = request.POST.get("discount")
    # 考虑库存的触发器条件
    package = request.POST.get("package")
    size = request.POST.get("size")
```

```

    pro_date = request.POST.get("pro_date")
    valid_date = request.POST.get("valid_date")
    models.drugs.objects.create(drug_name=drug_name, product_id=product_id,
product_place=product_place,
                                type=type, in_price=in_price,
single_price=single_price, discount=discount,
                                package=package, size=size, pro_date=pro_date,
valid_date=valid_date)
    return render(request, 'goods.html', {"info_dict": info_dict})

def delete_goods(request):
    id_dle = request.GET.get('id')
    print(id_dle)
    models.drugs.objects.filter(id=id_dle).delete()

    return redirect("/goods/")

def edit_goods(request):
    if request.method == "GET":
        id = request.GET.get("id")
        drug_obj = models.drugs.objects.filter(id=id).first()
        return render(request, 'edit_goods.html', {'drug_obj': drug_obj})
    else:
        member_id = request.GET.get("id")
        info_dict = request.session.get("info")
        print(info_dict)
        drug_name = request.POST.get("drug_name")
        print("drug_name", drug_name)
        product_id = request.POST.get("product_id")
        product_place = request.POST.get("product_place")
        type = request.POST.get("type")
        in_price = request.POST.get("in_price")
        single_price = request.POST.get("single_price")
        discount = request.POST.get("discount")
        package = request.POST.get("package")
        size = request.POST.get("size")
        pro_date = request.POST.get("pro_date")
        valid_date = request.POST.get("valid_date")
        models.drugs.objects.filter(id=member_id).update(drug_name=drug_name,
product_id=product_id,
                                                            product_place=product_place,
                                                            type=type, in_price=in_price,
single_price=single_price,
                                                            discount=discount,
                                                            package=package, size=size,
pro_date=pro_date,
                                                            valid_date=valid_date)

        return redirect("/goods/")

def in_out(request):
    queryset = models.in_out.objects.all()
    queryset1 = models.in_document.objects.all()

```

```

user_obj = models.user.objects.filter(user_name="lixuhui").first()
balance = user_obj.balance

    return render(request, 'in_out.html', {'queryset': queryset, 'queryset1':
queryset1, 'balance': balance})

def io_plot(request):
    # 绘图分析与结果
    legend = ["李旭辉", "李欣鸿"]
    date_list = ['1月', '2月', '3月', '4月', '5月', '6月']
    series_list = [
        {
            "name": '李旭辉',
            "type": 'bar',
            "data": [5, 20, 36, 10, 10, 20]
        },
        {
            "name": '李欣鸿',
            "type": 'bar',
            "data": [45, 20, 36, 10, 10, 100]
        }
    ]
    result = {
        "status": True,
        "data": {
            'legend': legend,
            'x_axis': date_list,
            'series_list': series_list
        }
    }
    return JsonResponse(result)

def add_in_out(request):
    if (request.method == "GET"):
        return render(request, 'in_out.html')
    info_dict = request.session.get("info")
    io_number = request.POST.get("收支编号")
    drug_id = request.POST.get("药品")
    user_increment = request.POST.get("关联用户编号")
    number = request.POST.get("数量")
    date = request.POST.get("日期")
    money = request.POST.get("总额")
    type = request.POST.get("类别")
    models.in_out.objects.create(io_number=io_number, drug_id=drug_id,
user_increment=user_increment,
                                number=number, date=date, money=money, type=type)
    return render(request, 'in_out.html', {"info_dict": info_dict})

def buying(request):
    id = request.GET.get("id")
    drug_obj = models.drugs.objects.filter(id=id).first()
    io_number = random.randint(1000, 2000)
    drug_id = drug_obj.product_id

```

```

number = 2
user_increment = 5
date = datetime.datetime.now()
price = models.drugs.objects.filter(id=id).first().single_price
# 更新余额
user_obj = models.user.objects.filter(user_name="lixuhui").first()
money_temp = user_obj.balance
money = -price * number + money_temp
models.user.objects.filter(user_name="lixuhui").update(balance=money)
# 成功修复总金额管理
# 更新库存
package_temp = drug_obj.package
package = package_temp - number
models.drugs.objects.filter(id=id).update(package=package)
type = drug_obj.type
models.in_out.objects.create(io_number=io_number, drug_id=drug_id,
user_increment=user_increment,
                             number=number, date=date, money=money, type=type)
queryset = models.in_out.objects.filter(io_number=io_number).first()
return render(request, 'buying.html', {'drug_obj': drug_obj, 'queryset':
queryset})

def number_revise(request):
    if request.method == "GET":
        id = request.GET.get("id")
        in_out_obj = models.in_out.objects.filter(id=id).first()
        return render(request, 'number_revise.html', {'in_out_obj': in_out_obj})
    else:
        in_out_id = request.GET.get("data")
        number = request.GET.get("number")
        models.in_out.objects.filter(id=in_out_id).update(id=in_out_id, number=number)
        return redirect("/buying/?id=13")

def delete_inout(request):
    id = request.GET.get('id')
    models.in_out.objects.filter(id=id).delete()
    return redirect("/inout/")

def returning_goods(request):
    id = request.GET.get("id")
    drug_obj = models.drugs.objects.filter(id=id).first()
    io_number = random.randint(1000, 2000)
    drug_id = drug_obj.product_id
    # user_increment = models.user.objects.filter(user_name="lixuhui").user_increment
    # user_increment = obj1.user_increment
    number = 1
    user_increment = 5
    date = datetime.datetime.now()
    type = -1
    # 更新余额
    price = drug_obj.single_price
    user_obj = models.user.objects.filter(user_name="lixuhui").first()

```



```

money_temp = user_obj.balance
money = price * number + money_temp
models.user.objects.filter(user_name="lixuhui").update(balance=money)
# 更新库存
package = drug_obj.package
models.drugs.objects.filter(id=id).update(package=package)
models.in_out.objects.create(io_number=io_number, drug_id=drug_id,
user_increment=user_increment,
                                number=number, date=date, money=money, type=type)
queryset = models.in_out.objects.filter(io_number=io_number).first()
return render(request, 'returning.html', {'drug_obj': drug_obj, 'queryset':
queryset})

def plotting(request):
    # 创建数据
    x = [1, 2, 3, 4, 5]
    y = [2, 4, 6, 6, 10]
    # 绘制折线图
    plt.plot(x, y)
    # 保存图像文件到指定路径
    save_path = '/static/chart1.png' # 替换为你想要保存的文件路径
    plt.savefig('chart.png')
    print("ok")
    return render(request, 'plotting.html')

def add_pro(request):
    if (request.method == "GET"):
        return render(request, 'add_provider.html')
    info_dict = request.session.get("info")
    provider_id = request.POST.get("provider_id")
    pro_name = request.POST.get("pro_name")
    linkman = request.POST.get("linkman")
    linkWay = request.POST.get("linkWay")
    city = request.POST.get("city")

    models.provider.objects.create(provider_id=provider_id, pro_name=pro_name,
linkman=linkman, linkWay=linkWay,
                                city=city)

    return render(request, 'depart.html', {"info_dict": info_dict})

def delete_pro(request):
    id = request.GET.get('id')
    print(id)
    models.provider.objects.filter(id=id).delete()
    return render(request, 'depart.html')

def edit_pro(request):
    if request.method == "GET":
        id = request.GET.get("id")
        provider_obj = models.provider.objects.filter(id=id).first()
        return render(request, 'edit_provider.html', {'provider_obj': provider_obj})

```

```

else:
    id = request.GET.get("id")
    provider_id = request.GET.get("provider_id")
    pro_name = request.GET.get("pro_name")
    linkman = request.GET.get("linkman")
    linkWay = request.GET.get("linkWay")
    city = request.GET.get("city")
    info_dict = request.session.get("info")
    models.provider.objects.filter(id=id).update(provider_id=provider_id,
    pro_name=pro_name, linkman=linkman,
                                                    linkWay=linkWay, city=city)

    return redirect("/depart/")

def add_worker(request):
    return render(request, 'add_worker.html')

def delete_worker(request):
    return render(request, 'add_worker.html')

def edit_worker(request):
    return render(request, 'add_worker.html')

def in_manage(request):
    queryset = models.drugs.objects.all()
    return render(request, 'in_manage.html', {'queryset': queryset})

def in_act(request):
    # queryset = models.in_document.objects.all()
    id = request.GET.get("id")
    drug_obj = models.drugs.objects.filter(id=id).first()
    provider_obj = models.provider.objects.filter(provider_id=5).first()
    in_out_obj = models.in_out.objects.last()
    in_id = random.randint(2000, 3000)
    provider_id = provider_obj.id
    name = drug_obj.drug_name
    io_number = in_out_obj.io_number
    number = 1
    # 进货数据更新
    user_obj = models.user.objects.filter(user_name="lixuhui").first()
    balance_temp = user_obj.balance
    price = drug_obj.single_price
    balance = balance_temp - price * number
    models.user.objects.filter(user_name="lixuhui").update(balance=balance)
    # 更新余额完成

    models.in_document.objects.create(in_id=in_id, provider_id=provider_id, name=name,
    io_number=io_number,
                                                    number=number)
    queryset = models.in_document.objects.filter(io_number=io_number).first()

```

```
    return render(request, 'in_act.html', {'drug_obj': drug_obj, 'queryset':
queryset})
```

```
def package(request):
    queryset = models.drugs.objects.all()
    return render(request, 'package.html', {'queryset': queryset})
```

model.py

```
from django.db import models
```

```
# 联表查询用数据库分隔开
```

```
# Create your models here.
```

```
class member(models.Model):
```

```
    id_client = models.CharField(verbose_name="客户编号", max_length=16)
```

```
    name = models.CharField(verbose_name="姓名", max_length=50)
```

```
    email = models.CharField(verbose_name="联系方式", max_length=32, null=True,
blank=True)
```

```
# models.member.create()
```

```
class provider(models.Model):
```

```
    provider_id = models.IntegerField(verbose_name="供应商编号")
```

```
    pro_name = models.CharField(verbose_name="供应商名称", max_length=50)
```

```
    linkman = models.CharField(verbose_name="联系人", max_length=50)
```

```
    linkWay = models.CharField(verbose_name="联系方式", max_length=50)
```

```
    city = models.CharField(verbose_name="城市", max_length=50)
```

```
class drugs(models.Model):
```

```
    drug_name = models.CharField(verbose_name="药品名称", max_length=50)
```

```
    # sup_id = models.IntegerField(verbose_name="供应商编号")
```

```
    # sup_id = models.ForeignKey(verbose_name="关联供应商", to="user",
on_delete=models.CASCADE)
```

```
    product_id = models.CharField(verbose_name="生产批号", max_length=100)
```

```
    product_place = models.CharField(verbose_name="产地", max_length=50)
```

```
    type = models.CharField(verbose_name="所属类别", max_length=50)
```

```
    in_price = models.FloatField(verbose_name="进价", max_length=10)
```

```
    single_price = models.FloatField(verbose_name="单价", max_length=10)
```

```
    discount = models.FloatField(verbose_name="会员折扣", max_length=3)
```

```
    package = models.IntegerField(verbose_name="库存")
```

```
    size = models.CharField(verbose_name="规格", max_length=50)
```

```
    pro_date = models.DateField(verbose_name="生产日期")
```

```
    valid_date = models.DateField(verbose_name="有效期")
```

```
class user(models.Model):
```

```
    user_increment = models.CharField(verbose_name="用户编号", max_length=500)
```

```
    user_name = models.CharField(verbose_name="用户名", max_length=50)
```

```
    pwd = models.CharField(verbose_name="密码", max_length=50)
```

```
    type = models.CharField(verbose_name="类别", max_length=50)
```

```
balance = models.IntegerField(verbose_name="您的余额")

class manager(models.Model):
    manager_increment = models.CharField(verbose_name="经理编号", max_length=500)
    # user_increment = models.CharField(verbose_name="用户编号", max_length=500)
    user_increment = models.ForeignKey(verbose_name="关联用户编号", to="user",
on_delete=models.CASCADE)

class worker(models.Model):
    worker_id = models.IntegerField(verbose_name="员工编号")
    worker_name = models.CharField(verbose_name="员工姓名", max_length=50)
    telephone = models.CharField(verbose_name="联系电话", max_length=500)
    # user_increment = models.CharField(verbose_name="用户编号", max_length=500)
    # 设置外码
    user_increment = models.ForeignKey(verbose_name="关联用户编号", to="user",
on_delete=models.CASCADE)

class in_out(models.Model):
    io_number = models.IntegerField(verbose_name="收支编号")
    drug_id = models.IntegerField(verbose_name="药品")
    # worker_id = models.IntegerField(verbose_name="员工编号", max_length=500)
    # user_increment = models.ForeignKey(verbose_name="关联用户编号", to="user",
on_delete=models.CASCADE, null=True, blank=True)
    user_increment = models.IntegerField(verbose_name="关联员工编号")
    number = models.IntegerField(verbose_name="数量")
    date = models.DateTimeField(verbose_name="日期")
    money = models.FloatField(verbose_name="总额", max_length=10)
    type = models.CharField(verbose_name="类别", max_length=50)

class in_document(models.Model):
    in_id = models.CharField(verbose_name="入库号", max_length=50)
    # provider_id = models.ForeignKey(verbose_name="管理供应商ID", to="provider",
on_delete=models.CASCADE)
    # io_number = models.ForeignKey(verbose_name="关联收支编号", to="in_out",
on_delete=models.CASCADE)
    provider_id = models.IntegerField(verbose_name="关联供应商ID")
    name = models.CharField(verbose_name="药品名称", max_length=50)
    io_number = models.IntegerField(verbose_name="关联收支编号")
    number = models.IntegerField(verbose_name="进货数量")

class sell_manage(models.Model):
    id_sell = models.IntegerField(verbose_name="销售编号")
    id_client = models.ForeignKey(verbose_name="关联客户编号", to="member",
on_delete=models.CASCADE)
    io_number = models.ForeignKey(verbose_name="关联收支编号", to="in_out",
on_delete=models.CASCADE)

class sales_returning(models.Model):
    id_returning = models.IntegerField(verbose_name="退货编号")
```

```

        id_sell = models.ForeignKey(verbose_name="关联销售编号", to="sell_manage",
on_delete=models.CASCADE)
        io_number = models.ForeignKey(verbose_name="关联收支编号", to="in_out",
on_delete=models.CASCADE)
        number = models.IntegerField(verbose_name="退货数量")

```

urls.py

```

from django.contrib import admin
from django.urls import path
from demo import views

urlpatterns = [
    path('admin/', admin.site.urls),
    path('index/', views.index),
    path('real/', views.real),
    path('depart/', views.depart),
    path('add/member/', views.add_member),
    path('delete/member/', views.delete_member),
    path('edit/member/', views.edit_member),
    path('goods/', views.goods),
    path('add/goods/', views.add_goods),
    path('delete/goods/', views.delete_goods),
    path('edit/goods/', views.edit_goods),
    path('add/user/', views.add_user),
    path('inout/', views.in_out),
    path('buying/', views.buying),
    path('delete/inout/', views.delete_inout),
    path('number/revise/', views.number_revise),
    path('returning/', views.returning_goods),
    path('plotting/', views.plotting),
    path('add/provider/', views.add_pro),
    path('delete/provider/', views.delete_pro),
    path('edit/provider/', views.edit_pro),
    path('add/worker/', views.add_worker),
    path('add/worker/', views.delete_worker),
    path('add/worker/', views.edit_worker),
    path('in_manage/', views.in_manage),
    path('in/act/', views.in_act),
    path('package/', views.package),
    path('io_plot/', views.io_plot),
]

```

setting.py

```

import os

# Build paths inside the project like this: os.path.join(BASE_DIR, ...)
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

# Quick-start development settings - unsuitable for production

```

```

# See https://docs.djangoproject.com/en/2.1/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = '@-+)!%9r&--bd0@i82a^gf1u=5!*0*7k=lv@yj$s1h41-*n='

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True

ALLOWED_HOSTS = []

# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'web.apps.WebConfig',
    'demo.apps.DemoConfig'
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    # 'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'ext.md.MyMiddlewareMixin',
]

ROOT_URLCONF = 'inven_manage.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        # 'DIRS': [os.path.join(BASE_DIR, 'static')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'inven_manage.wsgi.application'

```

```

# Database
# https://docs.djangoproject.com/en/2.1/ref/settings/#databases
# 默认数据库与我们需要的不一致
# DATABASES = {
#     'default': {
#         'ENGINE': 'django.db.backends.sqlite3',
#         'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),
#     }
# }
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'inventory_management_new',
        'USER': 'root',
        'PASSWORD': '123123',
        'HOST': '127.0.0.1',
        'PORT': 3306,
    }
}

# Password validation
# https://docs.djangoproject.com/en/2.1/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/2.1/topics/i18n/

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_L10N = True

USE_TZ = True

```

```
# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/2.1/howto/static-files/

STATIC_URL = '/static/'

SQL_FILE = [
    'demo/sql/custom.sql',
]
```

问题与解决方案

- 1.html语言使用是一大障碍，对于没有前端编程经验的我们来说完全是从零开始，解决方法就是搜索各种教程现学现用。
- 2.想要在页面中添加图片，但是加载不出来，仔细检查后发现，只要将图片引用为静态static{{}}即可。
- 3.向表格中添加表项时会出现外码关联错误的问题，是因为表格的参照完整性在一开始就定义好了，如果后来没有按规则修改，破坏了完整性，就会报错。解决方法就是在修改表格是需要谨慎，考虑到表格外码的定义。
- 4.执行购买操作后返回时界面错乱，系统bug，暂时无法解决。
- 5.管理员视图没有定义，数据库的连接性不可靠，由于时间关系暂时没有解决，需要更多时间修改。

总结与反思

本次实验难度较大，特别是在没有前端编程和PDBC编程经验的情况下。在紧张的开发期限中我们疯狂寻找各种教程和网课，最终跟着教程一步步敲代码，从零搭建起了一个还算美观的系统。添加一个个功能的过程就像是向着终点一步步奔跑，尽管很累，但也收获了很多，学到了很多。较为遗憾的是，关于视图部分完善度不高，外码关联存在错乱，后续我们会进一步修复。